Blocking and Nonblocking Assignments

RTL Design Guidelines

October 12, 2026

Contents

1 Introduction **2**

2 Blocking Assignment (=) **2**
2.1 Example 2
2.2 Guidelines 2

3 Nonblocking Assignment (<=) **2**
3.1 Example 3
3.2 Guidelines 3

4 Illustrative Example: Register Pipeline **3**

5 Why Blocking Assignments Can Cause Problems **4**

6 Comparison **4**

7 Professional RTL Coding Rules **4**

8 Summary **5**

1 Introduction

Verilog provides two commonly used procedural assignment operators:

- Blocking assignment: `=`
- Nonblocking assignment: `<=`

Choosing the correct assignment is important for accurate simulation, synthesizable RTL, and avoiding race conditions.

2 Blocking Assignment (=)

A blocking assignment updates the left-hand side immediately when the statement executes. The next statement in the same procedural block executes after that assignment.

2.1 Example

```
always @(*) begin
    temp = a & b;
    y    = temp | c;
end
```

Explanation:

- First, `temp` is assigned the value of `a & b`.
- Next, `y` is assigned `temp | c` using the updated value of `temp`.
- This coding style is suitable for combinational procedural logic.

2.2 Guidelines

- Prefer blocking assignments in combinational `always @(*)` blocks.
- Assign outputs on every possible execution path to avoid unintended latches.
- Use clear statement ordering when intermediate variables depend on previous assignments.
- Avoid using blocking assignments to model ordinary clocked register updates.

3 Nonblocking Assignment (<=)

A nonblocking assignment evaluates its right-hand side when the statement executes and schedules the left-hand-side update for the NBA region of the current simulation time slot.

3.1 Example

```
always @(posedge clk) begin
    if (rst)
        q <= 1'b0;
    else
        q <= d;
end
```

Explanation:

- The assignment is triggered by the rising edge of `clk`.
- The right-hand side is evaluated when the statement executes.
- The update to `q` is scheduled for the NBA region.
- This style models a clocked register.

3.2 Guidelines

- Prefer nonblocking assignments for clocked sequential logic.
- Use them consistently for registers updated in a clocked process.
- Avoid mixing blocking and nonblocking assignments to the same register.
- Do not depend on the execution order of independent clocked processes.

4 Illustrative Example: Register Pipeline

```
module pipeline (
    input  wire clk,
    input  wire d,
    output reg  q1,
    output reg  q2
);

    always @(posedge clk) begin
        q1 <= d;
        q2 <= q1;
    end

endmodule
```

At each rising clock edge:

- `q1` captures the current value of `d`.
- `q2` captures the previous value of `q1`.

Both right-hand sides are evaluated before their scheduled updates take effect. This correctly models two cascaded registers.

5 Why Blocking Assignments Can Cause Problems

Consider the following code:

```
always @(posedge clk) begin
    q1 = d;
    q2 = q1;
end
```

Because blocking assignments update immediately, `q2` receives the newly assigned value of `q1` in the same procedural execution.

This does not model two independent cascaded registers in the intended way.

The recommended version is:

```
always @(posedge clk) begin
    q1 <= d;
    q2 <= q1;
end
```

With nonblocking assignments, `q2` captures the previous value of `q1`.

6 Comparison

Feature	Blocking (=)	Nonblocking (<=)
Update	Immediate procedural update	Scheduled NBA update
Typical use	Combinational logic	Clocked sequential logic
Statement ordering	Later statements see earlier updates	Updates are deferred to the NBA region
Pipeline modeling	Can change intended behavior	Models cascaded registers correctly
Race risk	Possible when processes share data	Reduces common sequential race risks

7 Professional RTL Coding Rules

1. Use `=` for combinational procedural logic.
2. Use `<=` for clocked sequential logic.
3. Use complete assignments in combinational blocks to prevent unintended latches.
4. Avoid multiple procedural writers for the same register.
5. Avoid mixing assignment types for the same variable across procedural blocks.
6. Synchronize testbench stimulus and sampling to avoid race conditions.
7. Verify the design using simulation, lint, and synthesis checks.

8 Summary

- Blocking assignment (=) updates immediately and is generally preferred for combinational procedural logic.
- Nonblocking assignment (<=) schedules updates in the NBA region and is generally preferred for clocked sequential logic.
- Correct assignment styles help model intended hardware behavior and prevent simulation races.

Key rule: Use blocking assignments for combinational logic and nonblocking assignments for sequential logic.